

Node Types

Cache	Caching station
ClassSwitch	Class switching
Delay	Infinite server
Fork	Forks jobs
Join	Joins jobs
Logger	Trace logging
Place	Petri net place
Queue	Queueing station
Router	Routing node
Sink	Job departures (open)
Source	Job arrivals (open)
Transition	Petri net transition

R nodes are R6 objects: `Queue$new(model, "Q", SchedStrategy$FCFS)`. They register themselves with the model, so there is nothing to assign back.

Job Classes

`ClosedClass$new(model, "name", N, refStat)`
`OpenClass$new(model, "name")`
`SelfLoopingClass$new(model, "name", N, ref)`
`DisabledClass` has no R spelling; omit the class instead.

Distributions

<code>APH(α, T)</code>	Acyclic PH
<code>Cox2(μ, ϕ)</code>	Two-phase Coxian
<code>Coxian(μ, ϕ)</code>	Coxian
<code>Det(v)</code>	Deterministic
<code>Disabled</code>	None
<code>Erlang(λ, k)</code>	Erlang-k
<code>Exp(λ)</code>	Exponential
<code>Gamma(α, β)</code>	Gamma
<code>HyperExp(p, λ_1, λ_2)</code>	Hyper-exp
<code>Immediate</code>	Zero service time
<code>Lognormal(μ, σ)</code>	Log-normal
<code>Normal(μ, σ)</code>	Normal
<code>Pareto(α, k)</code>	Pareto
<code>PH(α, T)</code>	Phase-type
<code>Uniform(a, b)</code>	Uniform
<code>Weibull(α, β)</code>	Weibull

Discrete	
<code>Bernoulli(p)</code>	Bernoulli
<code>Binomial(p, n)</code>	Binomial
<code>DiscreteUniform(a, b)</code>	Discr. uniform
<code>Geometric(p)</code>	Geometric
<code>Poisson(λ)</code>	Poisson
<code>Zipf(s, n)</code>	Popularity

Correlated / trace-driven	
<code>BMAP(D0,...,Dk)</code>	Batch MAP
<code>NHPP(bp,λ,cyclic)</code>	Non-homog. Poisson
<code>DMAP(D0, D1)</code>	Discrete MAP
<code>EmpiricalCDF(F)</code>	Empirical CDF
<code>MAP(D0, D1)</code>	MAP
<code>MarkedMAP</code>	Marked MAP
<code>ME(α, T)</code>	Matrix-exp.
<code>CME(mean, order)</code>	Concentr. matrix-exp.
<code>MMPP2(λ_1, λ_2, σ_1, σ_2)</code>	MMPP(2)
<code>RAP(H0, H1)</code>	Rational AP
<code>RePlayer(trace)</code>	Trace replay

R distributions are plain functions, not classes: `Exp(0.5)`, `Erlang(2, 3)`. There is no `fitMean`; a

mean <i>m</i> is <code>Exp(1/m)</code> .	
Scheduling Strategies	
DPS	Discriminatory PS
EDD	Earliest Due Date
EDF	Earliest Deadline First
EXT	External (source)
FB	Feedback (LAS)
FCFS	First-Come First-Served
FCFSPI	FCFS Preempt. Resume-idle
FCFSPR	FCFS Preemptive Resume
FSP	Fair Sojourn Protocol
GPS	Generalized PS
HOL	Head-of-Line Priority
INF	Infinite Servers
LCFS	Last-Come First-Served
LCFSPI	LCFS Preempt. Resume-idle
LCFSPR	LCFS Preemptive Resume
LEPT	Longest Expected PT
LJF	Longest Job First
LPS	Limited PS
LRPT	Longest Remaining PT
OI	Order Independent
PAS	Pass and Swap
POLLING	Polling (switchover)
PS	Processor Sharing
PSJF	Preemptive SJF
SEPT	Shortest Expected PT
SETF	Shortest Elapsed Time First
SIRO	Random
SJF	Shortest Job First
SRPT	Shortest Remaining PT

Solvers

AUTO	Auto-select best
BA	Bound analysis (closed)
CTMC	CTMC (exact, small)
ENV	Random environments
FLD	Fluid/mean-field ODEs
LDES	Discrete Event Sim
LN	Layered networks
MAM	Matrix Analytic Methods
MVA	Mean Value Analysis
NC	Normalizing Constant Methods
SSA	Stochastic simulation

Wrappers

JMT	JMT simulator
LQNS	LQNS interface
QNS	LQNS product-form solver

Solver Options

<code>MVA\$new(model, "exact")</code>	
<code>JMT\$new(model, d = 123, samples = 10000)</code>	
<code>SSA\$new(model, samples = 5000, method = "para")</code>	
<code>LDES\$new(model, d = 23000)</code>	

Output Metrics

ArvR	Arrival rate
QLen	Mean queue length
ResidT	Residence time (total across visits)
RespT	Response time (per visit)
Tput	Throughput

Util	Server utilization
QLenVar	Queue-length variance
QLenSCV	Queue-length squared coeff. of variation
QLenSkew	Queue-length skewness
RespTVar	Response-time variance (FCFS only)
RespTSCV	Response-time SCV (FCFS only)
RespTSkew	Response-time skewness (FCFS only)

Moments (product form)

Moment tables have no R method; reach the engine's own report with `solver$analysis("var", raw = TRUE)`. *order* is a set: scalar *k* = up to *k*; a vector, e.g. `[2, 3]`, = exactly those. `RespT*` moments are FCFS-only (NaN elsewhere); exact methods cost $\prod_r (N_r + 1)$, use "lin" beyond that.

Analysis Methods

<code>solver\$avg_table()</code>	
<code>solver\$avg_sys_table()</code>	
<code>solver\$avg_node_table()</code>	
<code>solver\$avg_chain_table()</code>	
<code>solver\$get_avg()\$Qlen</code>	# matrices
<code>tget(tbl, "Qlen", "Q", "ci")</code>	

Table Methods

<code>avg_table</code>	per station
<code>avg_chain_table</code>	
	per chain
<code>avg_node_table</code>	per node
<code>avg_sys_table</code>	system
<code>get_avg</code>	as matrices
<code>analysis</code>	any –a name
<code>method_used</code>	what actually ran
<code>solver_meta</code>	method, arith, iter

Routing

<code>model\$link(serial_routing(n1, n2, n3))</code>	
<code>P <- model\$init_routing_matrix()</code>	
<code>P\$set(cls, n1, n2, 0.7)</code>	
<code>P\$set(cls, n1, n3, 0.3)</code>	
<code>model\$link(P)</code>	
<code>n1\$set_routing_strategy(cls, RoutingStrategy\$RROBIN)</code>	

Multi-Server & Finite Capacity

<code>queue\$set_number_of_servers(k)</code>	
<code>queue\$set_capacity(bufferSize)</code>	
<code>queue\$set_class_capacity(cls, n)</code>	
<code>queue\$set_drop_rule(cls, DropRule\$Drop)</code>	

Class Switching

<code>cs <- ClassSwitch\$new(model, "CS")</code>	
<code>P <- matrix(c(0.3, 0.7, 0.5, 0.5), nrow = 2, byrow = TRUE)</code>	
<code>cs\$set_class_switch_matrix(P)</code>	

Fork-Join Networks

Fork and Join have no R spelling. Build the model in another edition, save it, and `load_model()` it here to solve.

Layered Networks

R builds no LayeredNetwork. Solve an existing .lqn or layered document with `LN$new(load_model(path))`, or through `capi_run()` with `-i lqn`.

Caching

Cache nodes have no R spelling. Load a cache model written elsewhere and read its results with

`solver$analysis("cache")`.

ReplacementStrategy

CLIMB	Climb
FIFO	First-in first-out
HLRU	h-LRU
LRU	Least recently used
QLRU	q-LRU
RR	Random replacement
SFIFO	Static FIFO

Model Import/Export

<code>save_model(model, "model.json")</code>	
<code>model <- load_model("model.json")</code>	
<code>validate_model(model)</code>	
<code># no viewer; export and open in JMT</code>	

Routing Strategies

JSQ	Join shortest queue
KCHOICES	Power of k choices
PROB	Probabilistic routing
RAND	Random selection
RROBIN	Round-robin
WROBIN	Weighted round-robin

Stochastic Petri Nets

Places and Transitions have no R spelling. A Petri net built elsewhere still loads and solves through `load_model()`.

Markov Chains (solved directly)

MarkovProcess and MarkovChain have no R spelling. The engine's own routines are reachable by name: `api_invoke("ctmc_solve", list(Q = Q))`.

Random Environments

R builds no Environment. Solve one written elsewhere with `ENV$new(load_model(path))`.

Finite Capacity Regions

Finite capacity regions have no R spelling; per-station capacity does (`queue$set_capacity`, `set_class_capacity`, `set_drop_rule`).

Load-Dependent Service

<code>queue\$set_load_dependence(pmin(seq_len(N), c))</code>	
<code># class dependence takes no R callback</code>	

Impatience & Retrials

Patience, balking and retrial have no R spelling. Load a model that declares them and solve it as usual.

State Probabilities

<code>solver\$analysis("prob", raw = TRUE)</code>	
<code>solver\$analysis("marg", raw = TRUE)</code>	
<code>solver\$analysis("normconst", raw = TRUE)</code>	

Transient & Distributions

<code>solver\$analysis("tran", raw = TRUE)</code>	
<code>solver\$analysis("cdf", raw = TRUE)</code>	
<code>solver\$analysis("reward", raw = TRUE)</code>	
<code># rewards are declared in the document</code>	

Symbolic Analysis

<code># needs a SymEngine-enabled build</code>	
<code>line_version()\$features\$symengine</code>	
<code>solver\$analysis("sens", raw = TRUE)</code>	